VIBE CODING SPEC

Team Productivity Application

Collaborative Agile Workspace — Claude Code Ready Build Guide

Version 1.0 **Date** July 2026 **Status** Ready to Build **Stack** React 18 · Node.js 20 · PostgreSQL 16

Tasks 18 Claude Code Prompts

CONTENTS

1 Overview

A full-stack web application that integrates sprint task management with anonymous retrospective feedback for small agile teams of 4–8 people. A Project Manager registers, creates a team, and shares a 6-digit invite code with developers. The PM creates sprints, manages a backlog of tasks, and assigns them to the active sprint where developers move cards across a three-column Kanban board (To Do → In Progress → Done). When the PM clicks "Complete Sprint," the board locks, incomplete tasks return to the backlog, and an anonymous 3-question retrospective form is immediately unlocked for the entire team. The PM then views aggregated responses alongside the sprint's completed tasks — with no names attached — and closes the retro to enable the next sprint. Built on React, Node.js/Express, and PostgreSQL with JWT authentication; scoped to be completable as a student capstone project.

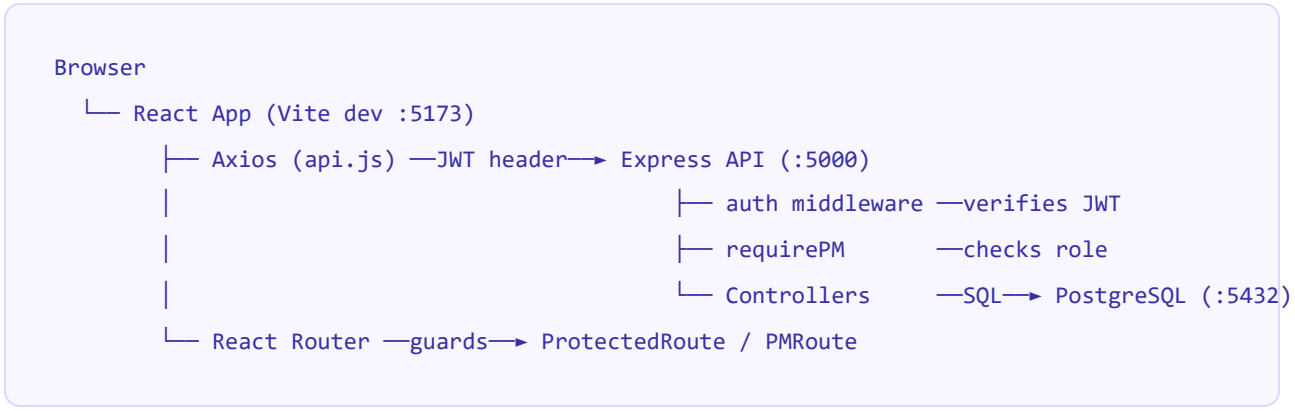
2 Tech Stack with Rationale

LAYER	TECHNOLOGY	VERSION	RATIONALE
Frontend Framework	React	18.3.x	Industry standard; component model maps cleanly to Kanban columns and modals

LAYER	TECHNOLOGY	VERSION	RATIONALE
Build Tool	Vite	5.x	Fast dev server; zero-config for React; significantly faster than Create React App
Routing	React Router DOM	6.x	Declarative route protection; nested routes for PM-only pages
HTTP Client	Axios	1.x	Interceptors allow JWT injection and 401-redirect in one place
Drag and Drop	@dnd-kit/core	6.x	Lightweight, accessible, actively maintained; purpose-built for lists and boards
Backend Runtime	Node.js	20 LTS	Same language as frontend; LTS stability for assignment duration
Backend Framework	Express	4.x	Minimal; middleware chain maps directly to auth → role-check → handler pattern
Database	PostgreSQL	16.x	Relational integrity enforces business rules at DB level (unique retro, sprint constraints)
DB Query Layer	node-postgres (pg)	8.x	Raw SQL — readable and debuggable; ORM abstraction not justified at this scale
Password Hashing	bcryptjs	2.x	Pure JS, no native bindings; bcrypt is correct for passwords
Auth Tokens	jsonwebtoken	9.x	Stateless JWT; no session store needed; 24h expiry suits team use
Rate Limiting	express-rate-limit	7.x	One-line brute-force protection; no Redis dependency needed
Environment Config	dotenv	16.x	Standard; keeps secrets out of source code
Dev Reload	nodemon	3.x	File-watch restart; standard in Node.js development

3 Architecture: Components and Interactions

SYSTEM INTERACTION FLOW



BACKEND COMPONENTS

FILE	RESPONSIBILITY
server/index.js	Boots Express; loads CORS, JSON parser, rate limiter; mounts all routers
server/config/db.js	Exports a single pg Pool; all controllers import from here
server/middleware/auth.js	Verifies JWT; attaches req.user = { id, name, role, teamId } to every protected request
server/middleware/requirePM.js	Returns 403 if req.user.role !== 'pm'; applied after auth middleware
server/middleware/rateLimiter.js	10 attempts per IP per 15 min on login → HTTP 429
server/routes/*.routes.js	One file per domain (auth, team, sprint, task, retro); mounts controllers
server/controllers/auth.controller.js	registerPM (creates team + user), registerDeveloper (validates invite), login
server/controllers/team.controller.js	Returns team info, member list, invite code
server/controllers/sprint.controller.js	Sprint CRUD + atomic complete transaction
server/controllers/task.controller.js	Task CRUD; status updates; sprintId assignment
server/controllers/retro.controller.js	Submit, status check, results (no user_id in output), close

FRONTEND COMPONENTS

FILE	RESPONSIBILITY
client/src/App.jsx	Route definitions; wraps app in AuthProvider

FILE	RESPONSIBILITY
client/src/context/AuthContext.jsx	JWT + user state; login/logout helpers; persists to localStorage
client/src/services/api.js	Axios instance; injects JWT header; redirects to /login on 401
client/src/components/ProtectedRoute.jsx	Redirects unauthenticated users to /login
client/src/components/PMRoute.jsx	Redirects non-PM users away from PM-only pages
client/src/pages/DashboardPage.jsx	**Data owner** for sprint + task state; delegates rendering to one of three child views
client/src/views/ActiveSprintView.jsx	Renders KanbanBoard; handles card-move callback → PATCH status
client/src/views/SprintCompleteView.jsx	Retro prompt banner + Done task list; links to retro form
client/src/views/NoSprintView.jsx	Empty-state with Create Sprint button (PM only)
client/src/components/KanbanBoard.jsx	**Controlled component** — receives tasks + onStatusChange as props; owns dnd-kit interaction only; never calls api.js directly
client/src/components/KanbanColumn.jsx	Single droppable column; task count badge; empty state
client/src/components/TaskCard.jsx	Draggable card; title, assignee initials, priority badge
client/src/components/TaskModal.jsx	Task detail/edit view; PM sees Edit/Delete; posts changes via parent callback
client/src/components/AddToSprintModal.jsx	Multi-select backlog tasks; confirm assigns sprintId via PATCH /tasks/:id
client/src/pages/BacklogPage.jsx	Lists backlog; opens TaskModal for create/edit; opens AddToSprintModal
client/src/pages/SprintCreatePage.jsx	Sprint creation form (PM only)
client/src/pages/RetroFormPage.jsx	3-question anonymous form; shows Done tasks as context above form
client/src/pages/RetroResultsPage.jsx	PM-only; left: Done tasks, right: grouped responses; Close Retro button
client/src/pages/SprintHistoryPage.jsx	PM-only list; click row → retro results (read-only)

FILE	RESPONSIBILITY
client/src/pages/TeamPage.jsx	PM-only; member table; copyable invite code

4 Data Model: Entities and Relationships

ENTITY RELATIONSHIPS

```
teams ──< users                (one team, many users)
teams ──< sprints              (one team, many sprints)
teams ──< tasks                (one team, many tasks)
sprints ──< tasks              (one sprint, many tasks — sprint_id nullable)
sprints ──< retro_submissions  (one sprint, many submissions)
users ──< tasks                (user assigned to task — assignee_id nullable)
users ──< retro_submissions    (one user, one submission per sprint)
```

COMPLETE SQL SCHEMA

```sql
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE teams (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name         VARCHAR(100) NOT NULL,
  invite_code  CHAR(6) UNIQUE NOT NULL,
  created_at   TIMESTAMP DEFAULT NOW()
);

CREATE TABLE users (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name           VARCHAR(100) NOT NULL,
  email          VARCHAR(255) UNIQUE NOT NULL,
  password_hash  TEXT NOT NULL,
  role           VARCHAR(20) NOT NULL CHECK (role IN ('pm', 'developer')),
  team_id        UUID REFERENCES teams(id),
  created_at     TIMESTAMP DEFAULT NOW()
);

CREATE TABLE sprints (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id           UUID NOT NULL REFERENCES teams(id),
  name              VARCHAR(100) NOT NULL,
```

```
  goal              TEXT,
  start_date        DATE NOT NULL,
  end_date          DATE NOT NULL,
  status            VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active','complete
  retro_closed      BOOLEAN DEFAULT FALSE,
  team_member_count INTEGER NOT NULL DEFAULT 0, -- snapshotted at sprint close
  created_at        TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_sprints_team_status ON sprints(team_id, status);

CREATE TABLE tasks (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id      UUID NOT NULL REFERENCES teams(id),
  sprint_id    UUID REFERENCES sprints(id), -- NULL = backlog
  title        VARCHAR(100) NOT NULL,
  description  TEXT,
  assignee_id  UUID REFERENCES users(id),
  priority     VARCHAR(10) DEFAULT 'medium' CHECK (priority IN ('low','medium','high'
  status       VARCHAR(20) DEFAULT 'backlog'
               CHECK (status IN ('backlog','todo','in_progress','done')),
  created_at   TIMESTAMP DEFAULT NOW(),
  updated_at   TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_tasks_sprint_id ON tasks(sprint_id);

CREATE TABLE retro_submissions (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sprint_id    UUID NOT NULL REFERENCES sprints(id),
  user_id      UUID NOT NULL REFERENCES users(id), -- stored but NEVER returned
  answer_1     TEXT NOT NULL,
  answer_2     TEXT NOT NULL,
  answer_3     TEXT NOT NULL,
  submitted_at TIMESTAMP DEFAULT NOW(),
  UNIQUE (sprint_id, user_id) -- one submission per user per sprint
);
CREATE INDEX idx_retro_sprint_id ON retro_submissions(sprint_id);
```

Tenancy rule: Every controller query on sprints, tasks, and retro_submissions must include `AND team_id = $req.user.teamId` in the WHERE clause to prevent cross-team data access.

5 API Design: Endpoints and Auth Requirements

METHOD	PATH	AUTH	ROLE	NOTES
POST	/api/auth/register	None	—	role in body decides PM vs Developer flow
POST	/api/auth/login	None	—	Rate-limited: 10 req / 15 min / IP
GET	/api/team	JWT	Any	Returns team info + member list
GET	/api/team/invite-code	JWT	PM	Returns invite code
GET	/api/sprints/active	JWT	Any	Returns sprint or null
GET	/api/sprints/history	JWT	PM	Completed sprints with stats
POST	/api/sprints	JWT	PM	Validates no active sprint / open retro
DELETE	/api/sprints/:id	JWT	PM	Blocked if sprint has tasks
PATCH	/api/sprints/:id/complete	JWT	PM	Atomic: status + snapshot + task reset
GET	/api/tasks/backlog	JWT	Any	Tasks with sprint_id IS NULL
GET	/api/tasks/sprint/:sprintId	JWT	Any	Tasks for a specific sprint
POST	/api/tasks	JWT	PM	Creates task in backlog or sprint
PATCH	/api/tasks/:id	JWT	PM	sprintId→UUID sets status=todo; sprintId→null sets status=backlog
PATCH	/api/tasks/:id/status	JWT	Any	Blocked if sprint is completed
DELETE	/api/tasks/:id	JWT	PM	Blocked if task has sprint_id
POST	/api/retro/:sprintId	JWT	Any	One submission per user; 409 on duplicate
GET	/api/retro/:sprintId/status	JWT	Any	Returns { submitted: boolean }
GET	/api/retro/:sprintId/results	JWT	PM	Responses grouped by question; user_id never returned
PATCH	/api/retro/:sprintId/close	JWT	PM	Sets retro_closed=true; unblocks new sprint

Error envelope: All errors return `{ "error": "Human-readable message.", "field": "fieldName" }` — `field` is only included for validation errors. Status codes: 400 validation, 401

unauthenticated, 403 forbidden, 404 not found, 409 conflict, 429 rate limited, 500 server error.

---

6 Implementation Tasks — Claude Code Prompts

> 18 prompts in strict dependency order. Each prompt is self-contained and paste-ready. Complete tasks in sequence — each assumes the previous is fully working. Phase labels: **Foundation** (Tasks 1–2), **Backend API** (Tasks 3–10), **Frontend** (Tasks 11–16), **Integration** (Task 17), **Polish** (Task 18).

01 Project Scaffolding

FOUNDATION

Claude Code Prompt — Task 01

PASTE READY

```
Create a full-stack project called team-productivity-app with this exact folder
structure:

team-productivity-app/
  server/
    config/
    controllers/
    middleware/
    routes/
    db/
    index.js
    package.json
    .env.example
  client/   ← initialize with Vite React template

In server/package.json, include these dependencies:
  express, pg, bcryptjs, jsonwebtoken, express-rate-limit, cors, dotenv
And devDependencies: nodemon

In server/.env.example:
  DATABASE_URL=postgresql://localhost:5432/team_productivity
  JWT_SECRET=your_jwt_secret_here
  PORT=5000
  CLIENT_URL=http://localhost:5173

In server/index.js, set up Express with:
  - cors middleware using CLIENT_URL from env (allow only that origin)
  - express.json() body parser
  - a health check: GET /api/health returns { "status": "ok" }
  - dev script in package.json: "dev": "nodemon index.js"

Initialize client/ with: npm create vite@latest client -- --template react
In client/package.json add: axios@1.x, react-router-dom@6.x, @dnd-kit/core@6.x

Do not create any routes, controllers, or database connection yet.
Print the full folder tree when done.
```

02 Database Schema and Seed Data FOUNDATION

Claude Code Prompt — Task 02 PASTE READY

```
In the team-productivity-app/server directory, do the following:

1. Create server/config/db.js that exports a pg Pool:
   import { Pool } from 'pg'
   import dotenv from 'dotenv'
   dotenv.config()
   const pool = new Pool({ connectionString: process.env.DATABASE_URL })
   export default pool

2. Create server/db/migrate.js that runs all CREATE TABLE statements when executed
   with "node server/db/migrate.js". Create these 5 tables in order:

   - teams: id (UUID PK), name (VARCHAR 100), invite_code (CHAR 6 UNIQUE), created_at
   - users: id (UUID PK), name, email (UNIQUE), password_hash, role CHECK IN
('pm','developer'),
            team_id FK → teams, created_at
   - sprints: id (UUID PK), team_id FK, name, goal, start_date DATE, end_date DATE,
              status CHECK IN ('active','completed') DEFAULT 'active',
              retro_closed BOOLEAN DEFAULT FALSE,
              team_member_count INTEGER NOT NULL DEFAULT 0, created_at
              + INDEX on (team_id, status)
   - tasks: id (UUID PK), team_id FK, sprint_id FK → sprints (nullable),
            title (VARCHAR 100), description TEXT, assignee_id FK → users (nullable),
            priority CHECK IN ('low','medium','high') DEFAULT 'medium',
            status CHECK IN ('backlog','todo','in_progress','done') DEFAULT
'backlog',
            created_at, updated_at
            + INDEX on sprint_id
   - retro_submissions: id (UUID PK), sprint_id FK, user_id FK,
                        answer_1 TEXT NOT NULL, answer_2 TEXT NOT NULL, answer_3 TEXT
NOT NULL,
                        submitted_at, UNIQUE(sprint_id, user_id)
                        + INDEX on sprint_id

   Use CREATE EXTENSION IF NOT EXISTS "pgcrypto" for gen_random_uuid().

3. Create server/db/seed.js that inserts test data:
   - Team: name='Test Team', invite_code='ABC123'
   - PM user: email='pm@test.com', password='password123' (bcrypt hash, 10 rounds),
role='pm'
   - Developer: email='dev@test.com', same password, role='developer', same team
   - Print "Seed complete" when done
```

```
Run migrate.js and seed.js and confirm both complete without errors.
```

03 Auth Middleware and Express Setup

BACKEND API

Claude Code Prompt — Task 03

PASTE READY

```
In team-productivity-app/server, create three middleware files.
Assume db.js exists at server/config/db.js and JWT_SECRET is in process.env.

1. server/middleware/auth.js
   - Extract Bearer token from Authorization header
   - Verify with jsonwebtoken using JWT_SECRET
   - If invalid or missing: return 401 { "error": "Authentication required." }
   - If valid: attach req.user = { id, name, email, role, teamId }
     where teamId comes from the 'team_id' field in the JWT payload
   - Call next()

2. server/middleware/requirePM.js
   - Runs after auth.js
   - If req.user.role !== 'pm': return 403 { "error": "Access denied." }
   - Otherwise call next()

3. server/middleware/rateLimiter.js
   - Use express-rate-limit
   - Max 10 requests per 15-minute window per IP
   - On exceed: return 429 { "error": "Too many login attempts. Try again later." }

4. Update server/index.js to:
   - Import rateLimiter but apply it only to POST /api/auth/login (not globally)
   - Keep auth and requirePM as exported modules — do not apply globally

Verify by adding a temporary test route that uses auth middleware, confirm it returns
401 with no token, then remove the test route.
```

04 Auth Endpoints: Register and Login

BACKEND API

Claude Code Prompt — Task 04

PASTE READY

```
In team-productivity-app/server, create the auth controller and route.

Create server/controllers/auth.controller.js with three exported functions:

1. registerPM(req, res)
   Validate: name (1-100 chars), email (valid format), password (8-128 chars)
   If email exists: return 400 { "error": "An account with this email already
exists." }
   Generate invite_code: 6 random uppercase alphanumeric chars via crypto.randomBytes
   In a single DB transaction:
     - INSERT INTO teams (name, invite_code) VALUES ($name + "'s Team", $code)
RETURNING id
     - Hash password with bcrypt (10 rounds)
     - INSERT INTO users (name, email, password_hash, role='pm', team_id)
   Sign JWT: payload = { id, name, email, role: 'pm', team_id } expires '24h'
   Return 201: { "token": "", "user": { "id","name","email","role":"pm","teamId" } }

2. registerDeveloper(req, res)
   Validate: name, email, password (same rules), inviteCode (6 alphanumeric chars)
   Look up team by invite_code. If not found: return 400 { "error": "Invalid invite
code. Please check with your Project Manager." }
   If email exists: return 400 { "error": "An account with this email already
exists." }
   Hash password, INSERT user with role='developer', team_id=found team id
   Sign JWT same as above with role: 'developer'
   Return 201: same shape

3. login(req, res)
   Validate: email and password present
   Fetch user by email. If not found OR bcrypt.compare fails:
     return 401 { "error": "Invalid email or password." }
   Sign JWT, return 200: { "token", "user": { id, name, email, role, teamId } }

Create server/routes/auth.routes.js:
   POST /api/auth/register — reads req.body.role to decide PM vs Developer handler
   POST /api/auth/login — apply rateLimiter middleware, call login handler

Mount in server/index.js: app.use('/api/auth', authRouter)

Test: register a PM, register a developer with returned invite code, log in as both.
```

05 **Team Endpoints** BACKEND API

Claude Code Prompt — Task 05 PASTE READY

```
In team-productivity-app/server, create team controller and route.
All queries must scope to req.user.teamId.

Create server/controllers/team.controller.js with:

1. getTeam(req, res)
   SELECT id, name, created_at FROM teams WHERE id = $req.user.teamId
   SELECT id, name, role, created_at FROM users WHERE team_id = $teamId ORDER BY
created_at ASC
   Return 200: { "team": { id, name, createdAt }, "members": [{ id, name, role,
createdAt }] }

2. getInviteCode(req, res)  ← PM only
   SELECT invite_code FROM teams WHERE id = $req.user.teamId
   Return 200: { "inviteCode": "ABC123" }

Create server/routes/team.routes.js:
   GET /api/team              → auth middleware → getTeam
   GET /api/team/invite-code  → auth middleware → requirePM → getInviteCode

Mount as app.use('/api/team', teamRouter)

Test both endpoints. Confirm /invite-code returns 403 for a developer token.
```

06 Task CRUD Endpoints

BACKEND API

Claude Code Prompt — Task 06 PASTE READY

```
In team-productivity-app/server, create the task controller and route.
ALL queries must include AND team_id = $req.user.teamId in WHERE clauses.

Create server/controllers/task.controller.js with six functions:

1. getBacklog(req, res)
   SELECT tasks.*, users.name as assignee_name, users.id as assignee_id
   FROM tasks LEFT JOIN users ON tasks.assignee_id = users.id
   WHERE tasks.team_id=$teamId AND tasks.sprint_id IS NULL ORDER BY created_at DESC
LIMIT 100
   Return 200: { "tasks": [{ id,title,description,priority,status,createdAt,
                             assignee: {id,name} | null }] }

2. getSprintTasks(req, res)
   Same JOIN but WHERE sprint_id=$sprintId AND tasks.team_id=$teamId
   Verify sprint belongs to team before querying.
   Return 200: same shape

3. createTask(req, res)  ← PM only
   Validate: title (1-100 chars required), description (max 1000, optional),
   priority ('low'|'medium'|'high', default 'medium'), assigneeId (UUID, optional),
   sprintId (UUID, optional)
   If sprintId: verify sprint belongs to team; set status='todo'. Else
status='backlog'
   INSERT and return 201: { "task": { ...all fields } }

4. updateTask(req, res)  ← PM only
   Accepts subset of: title, description, assigneeId, priority, sprintId
   If sprintId→UUID: set status='todo'. If sprintId→null: set status='backlog'
   UPDATE WHERE id=$id AND team_id=$teamId
   If no rows updated: 404. Return 200: { "task": { ...updated fields } }

5. updateTaskStatus(req, res)  ← any authenticated user
   Validate status IN ('todo','in_progress','done')
   Verify task belongs to team AND has sprint_id (not backlog)
   Fetch sprint — if status='completed': 403 { "error": "Sprint is closed. Board is
locked." }
   UPDATE SET status, updated_at=NOW()
   Return 200: { "task": { id, status, updatedAt } }

6. deleteTask(req, res)  ← PM only
   If task has sprint_id: 409 { "error": "Cannot delete a task that is in a sprint."
}
   DELETE WHERE id=$id AND team_id=$teamId. Return 204.

Create routes with auth + PM guards. Mount as app.use('/api/tasks', taskRouter).
```

```
Test all 6 ops including: create in backlog, assign to sprint via updateTask, move
status,
attempt delete of sprint task (expect 409).
```

07 Sprint CRUD Endpoints (Excluding Complete) BACKEND API

Claude Code Prompt — Task 07 PASTE READY

```
In team-productivity-app/server, create sprint controller (partial) and route.
Do NOT implement the complete endpoint yet — that is Task 09.

Create server/controllers/sprint.controller.js with:

1. getActiveSprint(req, res)
   SELECT * FROM sprints WHERE team_id=$teamId AND status='active' LIMIT 1
   Return 200: { "sprint": { id,name,goal,startDate,endDate,status,retroClosed } }
   If none: return 200: { "sprint": null }

2. getSprintHistory(req, res)  ← PM only
   SELECT s.id, s.name, s.start_date, s.end_date,
     COUNT(t.id) as tasks_total,
     COUNT(t.id) FILTER (WHERE t.status='done') as tasks_done,
     s.team_member_count as retro_total,
     COUNT(rs.id) as retro_submitted
   FROM sprints s
   LEFT JOIN tasks t ON t.sprint_id=s.id
   LEFT JOIN retro_submissions rs ON rs.sprint_id=s.id
   WHERE s.team_id=$teamId AND s.status='completed'
   GROUP BY s.id ORDER BY s.end_date DESC LIMIT 50
   Return 200: { "sprints": [{
id,name,startDate,endDate,tasksTotal,tasksDone,retroTotal,retroSubmitted }] }

3. createSprint(req, res)  ← PM only
   Validate: name (1-100 required), endDate (valid date, today or future), goal
(optional)
   Check: if any active sprint exists for team → 409 { "error": "A sprint is already
active." }
   Check: if any completed sprint with retro_closed=false → 409 { "error": "Close the
current retrospective before starting a new sprint." }
   INSERT with start_date=CURRENT_DATE. Return 201: { "sprint": { all fields } }

4. deleteSprint(req, res)  ← PM only
   Fetch sprint — if not found: 404
   If COUNT(tasks WHERE sprint_id=$id) > 0 → 409 { "error": "Remove all tasks before
deleting." }
   DELETE, return 204

Create routes, mount as app.use('/api/sprints', sprintRouter).
Test: create sprint, attempt second (expect 409), delete empty sprint.
```

08 Retro Submit and Status Endpoints

BACKEND API

Claude Code Prompt — Task 08

PASTE READY

```
In team-productivity-app/server, create the retro controller (partial).
Implement only submit and status — results and close come in Task 10.

Create server/controllers/retro.controller.js with:

1. submitRetro(req, res)
   Validate :sprintId is UUID. Fetch sprint WHERE id=$sprintId AND team_id=$teamId.
   If not found: 404.
   If sprint.status !== 'completed': 403 { "error": "Retrospective is not open yet." }
   Validate body: answer_1, answer_2, answer_3 — each required, min 10 chars, max 500 chars
   If field fails: 400 { "error": "...", "field": "answer_1" } (or answer_2/answer_3)
   INSERT INTO retro_submissions (sprint_id, user_id, answer_1, answer_2, answer_3)
   If UNIQUE constraint violation: 409 { "error": "You have already submitted feedback for this sprint." }
   Return 201: { "message": "Submission recorded." }
   CRITICAL: user_id stored in DB but must NEVER appear in any response body.

2. getRetroStatus(req, res)
   Verify sprint belongs to team.
   SELECT COUNT(*) FROM retro_submissions WHERE sprint_id=$id AND user_id=$req.user.id
   Return 200: { "submitted": boolean }

Create server/routes/retro.routes.js (partial):
   POST /api/retro/:sprintId          → auth → submitRetro
   GET  /api/retro/:sprintId/status   → auth → getRetroStatus

Mount as app.use('/api/retro', retroRouter).
Test: complete a sprint manually in DB, submit as PM and developer,
attempt double-submit (expect 409). Verify user_id absent from all responses.
```

09 Sprint Complete — Atomic Transaction

BACKEND API

Claude Code Prompt — Task 09

PASTE READY

```
In team-productivity-app/server, add the complete sprint endpoint to the existing
sprint controller. This is the most critical endpoint — implement carefully.

Add completeSprint(req, res) to server/controllers/sprint.controller.js:

1. Fetch sprint WHERE id=$id AND team_id=$teamId. If not found: 404.
   If sprint.status === 'completed': 409 { "error": "This sprint is already
completed." }

2. Get current team member count:
   SELECT COUNT(*) as count FROM users WHERE team_id=$teamId

3. Begin a database transaction using pool.connect() + client.query('BEGIN'):
   a. UPDATE sprints SET status='completed', team_member_count=$count WHERE id=$id
   b. UPDATE tasks SET status='backlog', sprint_id=NULL, updated_at=NOW()
      WHERE sprint_id=$id AND status IN ('todo','in_progress')
      RETURNING id   ← count returned rows as tasksReturned
   c. COMMIT

4. If any step throws: ROLLBACK, return 500 { "error": "Failed to complete sprint." }

5. Return 200: { "sprint": { id, status: "completed" }, "tasksReturned": number }

Add to server/routes/sprint.routes.js:
   PATCH /api/sprints/:id/complete → auth → requirePM → completeSprint

Test the full sequence:
  1. Create sprint, add 3 tasks, move 2 to done, leave 1 in_progress
  2. Call PATCH /api/sprints/:id/complete
  3. Verify: sprint status=completed, in_progress task returned to backlog with
sprint_id=NULL,
     done tasks still attached to sprint with status=done
  4. Verify: team_member_count correctly set on sprint row
  5. Attempt to complete same sprint again (expect 409)
  6. Attempt PATCH /api/tasks/:id/status on completed sprint (expect 403)
```

10 Retro Results and Close Endpoints

BACKEND API

Claude Code Prompt — Task 10 PASTE READY

```
In team-productivity-app/server, complete the retro controller and route.

Add to server/controllers/retro.controller.js:

1. getRetroResults(req, res)  ← PM only
   Fetch sprint WHERE id=$sprintId AND team_id=$teamId. If not found: 404.

   Query 1 — participation:
     SELECT COUNT(DISTINCT user_id) as submitted FROM retro_submissions WHERE
sprint_id=$id
     Use sprint.team_member_count as total

   Query 2 — completed tasks:
     SELECT t.id, t.title, t.priority, u.name as assignee_name
     FROM tasks t LEFT JOIN users u ON t.assignee_id=u.id
     WHERE t.sprint_id=$id AND t.status='done'

   Query 3 — responses (CRITICAL: do NOT select user_id):
     SELECT answer_1, answer_2, answer_3 FROM retro_submissions WHERE sprint_id=$id

   Build response object:
     responses.question1 = all answer_1 values as array
     responses.question2 = all answer_2 values as array
     responses.question3 = all answer_3 values as array

   Return 200:
   {
     "sprint": { id, name, retroClosed },
     "participation": { submitted: number, total: number },
     "tasks": [{ id, title, priority, assignee: {name} | null }],
     "responses": { question1: [...], question2: [...], question3: [...] }
   }

2. closeRetro(req, res)  ← PM only
   Fetch sprint. If not found: 404.
   If sprint.retro_closed: 409 { "error": "Retrospective is already closed." }
   UPDATE sprints SET retro_closed=true WHERE id=$id AND team_id=$teamId
   Return 200: { "message": "Retrospective closed." }

Add to routes:
   GET   /api/retro/:sprintId/results  → auth → requirePM → getRetroResults
   PATCH /api/retro/:sprintId/close    → auth → requirePM → closeRetro

Test: submit retro as 2+ users, GET results as PM (verify no user_id anywhere in
```

```
response),
call close, then verify createSprint now succeeds.
```

11 React App Setup: Routing, Auth Context, API Service FRONTEND

Claude Code Prompt — Task 11 PASTE READY

```
In team-productivity-app/client/src, set up the React foundation.
Vite React template is initialized. React Router DOM and Axios are installed.

1. Create client/src/context/AuthContext.jsx:
   - Context with createContext()
   - AuthProvider holds state: { user, token } (null initially)
   - On mount: read token from localStorage, decode payload with
     JSON.parse(atob(token.split('.')[1])) to restore user state
   - login(token): save to localStorage, decode and set user state
   - logout(): remove from localStorage, clear state
   - Export useAuth() returning { user, token, login, logout, isAuthenticated: !!user
}

2. Create client/src/services/api.js:
   - Axios instance with baseURL = import.meta.env.VITE_API_URL ||
'http://localhost:5000'
   - Request interceptor: read token from localStorage, add Authorization: Bearer
header
   - Response interceptor: if status===401, clear localStorage + redirect
     window.location.href = '/login?reason=expired'
   - Export as default

3. Create client/src/components/ProtectedRoute.jsx:
   - Uses useAuth(). If !isAuthenticated:
   - Otherwise:  (React Router v6)

4. Create client/src/components/PMRoute.jsx:
   - If !isAuthenticated:
   - If user.role !== 'pm':
   - Otherwise:

5. Create client/src/App.jsx with routes:
   Public:          /login,  /register
   Protected (any): /dashboard,  /retro/:sprintId
   PM-only:         /backlog,  /sprint/new,  /retro/:sprintId/results,  /history,
/team
   Redirect:        / → /dashboard
   Wrap in . Use  and  wrapper elements.

6. Create placeholder pages (return
PageName
) for: LoginPage, RegisterPage,
   DashboardPage, BacklogPage, SprintCreatePage, RetroFormPage, RetroResultsPage,
   SprintHistoryPage, TeamPage

7. Create client/.env: VITE_API_URL=http://localhost:5000
```

```
Run dev server. Confirm all routes render. Confirm ProtectedRoute redirects to
/login.
```

12 Login and Register Pages

FRONTEND

Claude Code Prompt — Task 12

PASTE READY

```
In team-productivity-app/client/src/pages, build LoginPage and RegisterPage.

1. LoginPage.jsx:
   - Form: email, password fields
   - On submit: POST /api/auth/login with { email, password }
   - On success: call login(token) from useAuth(), navigate to /dashboard
   - On 401: show error below form: "Invalid email or password."
   - On 429: show error: "Too many login attempts. Please try again later."
   - If URL has ?reason=expired: show banner at top: "Session expired. Please log in
again."
   - Show loading state on submit button while request is in flight
   - Client validation: both fields required before submitting

2. RegisterPage.jsx:
   - Form: name, email, password, role (radio: 'pm' | 'developer')
   - When role='developer': show inviteCode field. When role='pm': hide it completely
   - On submit: POST /api/auth/register with all fields
   - On success: call login(token), navigate to /dashboard
   - Show field-level errors for 400 responses using "field" key from error response
   - Show general error for 409: "An account with this email already exists."
   - Client validation before submit:
     - name: required
     - email: valid format
     - password: min 8 characters
     - inviteCode: required, exactly 6 chars when role=developer
   - Show loading state on button

Test: register a PM, register a developer with the invite code, log in as both.
Confirm redirect to /dashboard after each successful auth.
```

13 Dashboard Page with Three View States

FRONTEND

Claude Code Prompt — Task 13 PASTE READY

```
In team-productivity-app/client/src, build DashboardPage and three child views.
DashboardPage is the single data owner — it fetches all sprint/task data.

1. DashboardPage.jsx (src/pages/):
   On mount, fetch in parallel:
     - GET /api/sprints/active → sprint
     - If sprint exists: GET /api/tasks/sprint/:id → tasks
   State: { sprint, tasks, loading, error }

   Render logic:
     - loading: centered spinner
     - sprint===null:
     - sprint.status==='active':
     - sprint.status==='completed' && !sprint.retroClosed:
     - sprint.retroClosed:  (ready for next sprint)

   handleStatusChange(taskId, newStatus):
     PATCH /api/tasks/:taskId/status with { status: newStatus }
     On success: update tasks array immutably in state
     On error: revert task to original status, show toast: "Failed to update task."

   handleSprintComplete():
     Re-fetch GET /api/sprints/active and update sprint in state

2. ActiveSprintView.jsx (src/views/):
   Props: { sprint, tasks, onTaskStatusChange, onSprintComplete }
   - Sprint name + end date as header
   -
   - If user.role==='pm': "Complete Sprint" button
   - On click: show confirmation modal with count of incomplete tasks:
     "X tasks are still incomplete and will return to backlog."
     If 0: "All tasks are complete. Ready to close?"
     Buttons: Cancel and Complete Sprint
   - On confirm: PATCH /api/sprints/:id/complete, on success call onSprintComplete()

3. SprintCompleteView.jsx (src/views/):
   Props: { sprint }
   - Banner: "Sprint [name] is complete. Share your feedback."
   - Fetch done tasks for context (GET /api/tasks/sprint/:id, filter status=done)
   - Button link to /retro/:sprintId
   - If user.role==='pm': also show link to /retro/:sprintId/results

4. NoSprintView.jsx (src/views/):
   - "No active sprint."
   - If user.role==='pm': "Create Sprint" button → navigate to /sprint/new
```

```
Test all three view states by setting sprint status directly in DB.
```

14 Kanban Board with Drag and Drop

FRONTEND

Claude Code Prompt — Task 14 PASTE READY

```
In team-productivity-app/client/src/components, build the Kanban board.
This is a CONTROLLED component — receives tasks and onStatusChange as props.
It must NEVER call api.js directly.

1. KanbanBoard.jsx:
   Props: { tasks: Task[], onStatusChange: (taskId, newStatus) => void, locked:
boolean }
   - Use @dnd-kit/core: DndContext, DragOverlay, useSensor, PointerSensor
   - Internal state: activeTask (card being dragged)
   - Three columns: 'todo', 'in_progress', 'done'
   - Filter tasks prop into each column by status field
   - If locked=true: disable dnd sensors; show locked overlay over entire board
   - On drag end: determine destination column, call onStatusChange(taskId,
columnStatus)
   - Render three  components

2. KanbanColumn.jsx:
   Props: { id, title, tasks, onCardClick }
   - Use @dnd-kit useDroppable
   - Column title + task count badge (e.g. "To Do  3")
   - If tasks.length===0: show "No tasks here." placeholder
   - Render  for each task

3. TaskCard.jsx:
   Props: { task, onClick }
   - Use @dnd-kit useDraggable
   - Show: title (truncate at 60 chars), assignee initials in colored circle (grey if
unassigned),
     priority badge (low=green, medium=yellow, high=red background)
   - On click (not drag): call onClick(task)

4. TaskModal.jsx:
   Props: { task: Task | null, onClose, onUpdate, onDelete }
   - If task is null: render nothing
   - Show: title, description, assignee, priority, status as read-only
   - If user.role==='pm': show Edit and Delete buttons
   - Edit mode: inline form; on Save: PATCH /api/tasks/:id; call
onUpdate(updatedTask)
   - Delete: confirm prompt; DELETE /api/tasks/:id; call onDelete(task.id)
   - Close on backdrop click or X button

Wire TaskCard onClick to open TaskModal in ActiveSprintView.
Test dragging between all columns. Verify locked=true prevents dragging.
```

15 Backlog Page with Task and Sprint Modals

FRONTEND

Claude Code Prompt — Task 15

PASTE READY

```
In team-productivity-app/client/src/pages, build BacklogPage (PM-only).

1. BacklogPage.jsx:
   On mount fetch in parallel:
     - GET /api/tasks/backlog → tasks state
     - GET /api/team → members state (for assignee dropdown in TaskModal)
     - GET /api/sprints/active → activeSprint state
   Render:
   - "Backlog" heading + task count
   - "Add Task" button (PM only) → opens TaskModal in create mode
   - Task list: each row shows title, priority badge, assignee name, created date
   - Each row has Edit (pencil) and Delete (trash) icon buttons
   - "Add to Sprint" button only when activeSprint exists → opens AddToSprintModal
   - Empty state: "No tasks in backlog. Create a task to get started."

2. AddToSprintModal.jsx:
   Props: { tasks, sprintId, onClose, onAdded }
   - Checkbox list of backlog tasks
   - "Add X tasks to sprint" button (disabled when 0 selected)
   - On confirm: PATCH /api/tasks/:id with { sprintId } for each selected task
     Run all PATCHes in parallel with Promise.all
   - On all complete: call onAdded() (parent refreshes backlog)

3. Extend TaskModal.jsx to support create mode:
   When no task prop (create mode): show empty form with all fields
   On save in create mode: POST /api/tasks
   On success: call onUpdate(newTask) (parent adds to list)

Test: create tasks, edit, delete, add multiple tasks to sprint in one action.
Confirm deleted tasks with sprint_id are blocked (expect API 409).
```

16 Remaining Pages: Sprint Create, Retro Form, Retro Results, History, Team FRONTEND

Claude Code Prompt — Task 16 PASTE READY

```
In team-productivity-app/client/src/pages, build the remaining 5 pages.

1. SprintCreatePage.jsx (PM only):
   Form: name (required, max 100), endDate (required, YYYY-MM-DD, not in past), goal
(optional)
   POST /api/sprints on submit. On success: navigate to /dashboard
   On 409 "active sprint": "A sprint is already active."
   On 409 "retro open": "Close the current retrospective first."

2. RetroFormPage.jsx (all authenticated):
   On mount:
     - GET /api/retro/:sprintId/status → if submitted=true, show confirmation state
immediately
     - GET /api/tasks/sprint/:sprintId filtered to done tasks → show as context above
form
   Form: three labeled textareas:
     Q1: "What went well this sprint?"
     Q2: "What didn't go well?"
     Q3: "What should we improve next sprint?"
   Each: min 10 chars, max 500 chars, show character counter
   On submit: POST /api/retro/:sprintId with { answer1, answer2, answer3 }
   On success or 409: show "Your feedback has been submitted. Thank you."

3. RetroResultsPage.jsx (PM only):
   Fetch GET /api/retro/:sprintId/results on mount
   Two-column layout:
     LEFT: "Completed Tasks" — list of done tasks (title, priority badge, assignee)
     RIGHT: For each of 3 questions, show heading + all responses as cards (no names)
   Top: "X of Y team members responded"
   Bottom: "Close Retro" button → confirm prompt → PATCH /api/retro/:sprintId/close
   On success: show "Retro closed." + "Create New Sprint" button → /sprint/new

4. SprintHistoryPage.jsx (PM only):
   Fetch GET /api/sprints/history on mount
   Table: Sprint Name, Dates, Completed Tasks (X/Y), Retro Responses (X/Y)
   Each row clickable → /retro/:sprintId/results
   Empty state: "No completed sprints yet."

5. TeamPage.jsx (PM only):
   Fetch GET /api/team and GET /api/team/invite-code on mount
   Show invite code prominently with "Copy" button (copies to clipboard, shows
"Copied!" for 2s)
   Member table: Name, Role, Joined date
```

```
Test all 5 pages for data loading, PM-only access, and empty states.
```

17 Full API Integration and End-to-End Testing

INTEGRATION

Claude Code Prompt — Task 17 PASTE READY

```
Start both server (port 5000) and client (port 5173). Run this 10-step demo flow
and fix any issues found before declaring integration complete.

STEP 1: /register as PM (name, email, password)
  → Confirm team created, redirect to /dashboard showing NoSprintView with "Create
Sprint" button

STEP 2: Copy invite code from /team
  → Open incognito, register as Developer with code
  → Confirm redirect to /dashboard (same view, no Create Sprint button)

STEP 3: As PM → /backlog, create 4 tasks with different priorities and assignees

STEP 4: As PM → /sprint/new, create "Sprint 1" with future end date
  → Confirm redirect to /dashboard showing KanbanBoard with tasks in To Do column

STEP 5: As Developer, refresh /dashboard
  → Developer drags 3 tasks to Done, leaves 1 in In Progress
  → As PM, refresh — confirm card positions updated

STEP 6: As PM, click "Complete Sprint"
  → Confirm dialog shows correct incomplete task count
  → Complete sprint → dashboard shows SprintCompleteView

STEP 7: Both PM and Developer navigate to /retro/:sprintId
  → Both submit all 3 answers
  → Attempt re-submission as same user → confirm "already submitted" state

STEP 8: As PM → /retro/:sprintId/results
  → Confirm "2 of 2 members responded"
  → Confirm left panel shows done tasks, right panel shows responses with NO names
  → Click "Close Retro" → success + "Create New Sprint" button

STEP 9: /history → Sprint 1 shows with correct task and retro stats

STEP 10: Click Sprint 1 row → retro results open in read-only mode (no Close button)

Also verify:
  - Expire a JWT manually in DB/env, make a request, confirm redirect to /login?
reason=expired
  - As Developer, navigate directly to /team → confirm redirect to /dashboard

Fix every failing step before Task 18.
```

18 Validation, Empty States, Error Handling, and Polish POLISH

Claude Code Prompt — Task 18 PASTE READY

```
In team-productivity-app/client, complete all remaining polish items.

1. CLIENT-SIDE VALIDATION (show inline errors, prevent submission):
   - LoginPage: email required + valid format, password required
   - RegisterPage: name 1-100 chars, email valid, password min 8, inviteCode 6
alphanumeric if developer
   - SprintCreatePage: name required, endDate required and not in past
   - TaskModal create/edit: title required 1-100 chars, description max 1000 chars
   - RetroFormPage: all 3 answers required, min 10 chars, max 500 chars with counter

2. LOADING STATES (spinner/skeleton while fetching):
   - DashboardPage: full-page spinner while loading active sprint
   - BacklogPage: skeleton rows while loading
   - RetroResultsPage: spinner while loading results
   - SprintHistoryPage: spinner while loading history
   - All submit buttons: disabled + "Loading..." text while request is in flight

3. EMPTY STATES for all list views:
   - Backlog: "No tasks in backlog. Create a task to get started."
   - Kanban column: "No tasks here."
   - Sprint history: "No completed sprints yet."
   - Retro results 0 responses: "No responses submitted yet."
   - Retro results 0 done tasks: "No tasks were completed in this sprint."

4. TOAST NOTIFICATIONS (top-right, disappear after 3 seconds):
   - Card move failure: "Failed to update task. Please try again." (red)
   - Invite code copied: "Copied!" (green)
   - Retro closed: "Retrospective closed successfully." (green)
   - Unexpected 500: "Something went wrong. Please try again." (red)

5. SECURITY VERIFICATION (check these manually in DevTools):
   - Submit retro, inspect network response — confirm user_id is absent everywhere
   - As Developer, navigate to /retro/:sprintId/results — confirm redirect
   - As Developer, fetch GET /api/team/invite-code from console → confirm 403
   - Expire JWT, attempt card move → confirm redirect to /login?reason=expired

The app is complete when all 10 steps from Task 17 pass and all 5 items above are
done.
```

7 Out of Scope

Features not listed in this spec and not listed below should default to out of scope.

USER & ACCOUNT MANAGEMENT

Password reset, email verification, profile editing, account deletion, PM role transfer, removing team members, team deletion, multiple teams per user

TASK & SPRINT MANAGEMENT

Sprint editing after creation, sprint reopening, multiple concurrent sprints, subtasks, task dependencies, task due dates, task comments, file attachments, search/filter/sort, reordering within column, story points, burndown charts

RETROSPECTIVE

Custom questions, alternative formats, voting on responses, grouping/theming responses, action item tracking, editing/deleting submissions, developer access to results, sentiment analysis, auto-close

NOTIFICATIONS & INTEGRATIONS

Email of any kind, in-app notification centre, push notifications, Jira/Linear/GitHub import, commit linking, Slack/Teams integration, OAuth/SSO login, outbound webhooks, calendar integrations

PLATFORM & COMPLIANCE

WebSocket/real-time sync, mobile app, mobile-responsive design, WCAG accessibility compliance, internationalisation, dark mode, two-factor auth, audit logging, GDPR tooling, data export

8 Open Questions

#	QUESTION	IMPACT IF UNRESOLVED
OQ-01	Should the retro auto-close after 7 days if the PM never manually closes it?	Without a limit, an unclosed retro permanently blocks all new sprint creation for that team
OQ-02	After submitting the retro, should a Developer see the list of Done tasks from the completed sprint?	Currently no — but showing tasks (not responses) may improve transparency without breaking anonymity

#	QUESTION	IMPACT IF UNRESOLVED
OQ-03	Should a team member who joins after a sprint closes be counted in the "Y of Y responded" total?	Currently excluded — team_member_count is snapshotted at sprint close, so late joiners don't inflate the denominator